KDRSolvers:

Scalable, Flexible, Task-Oriented Krylov Solvers

PAW-ATM 2025
David K. Zhang, Rohan Yadav, Alex Aiken, Fred Kjolstad, Sean Treichler
Stanford University

$A\mathbf{x} = \mathbf{b}$ Done Right

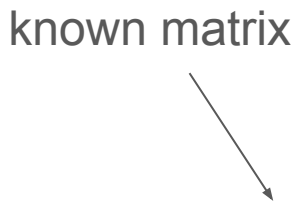
PAW-ATM 2025

David K. Zhang, Rohan Yadav, Alex Aiken, Fred Kjolstad, Sean Treichler
Stanford University

$$A\mathbf{x} = \mathbf{b}$$

$$\mathbf{A}\mathbf{x} = \mathbf{b}$$

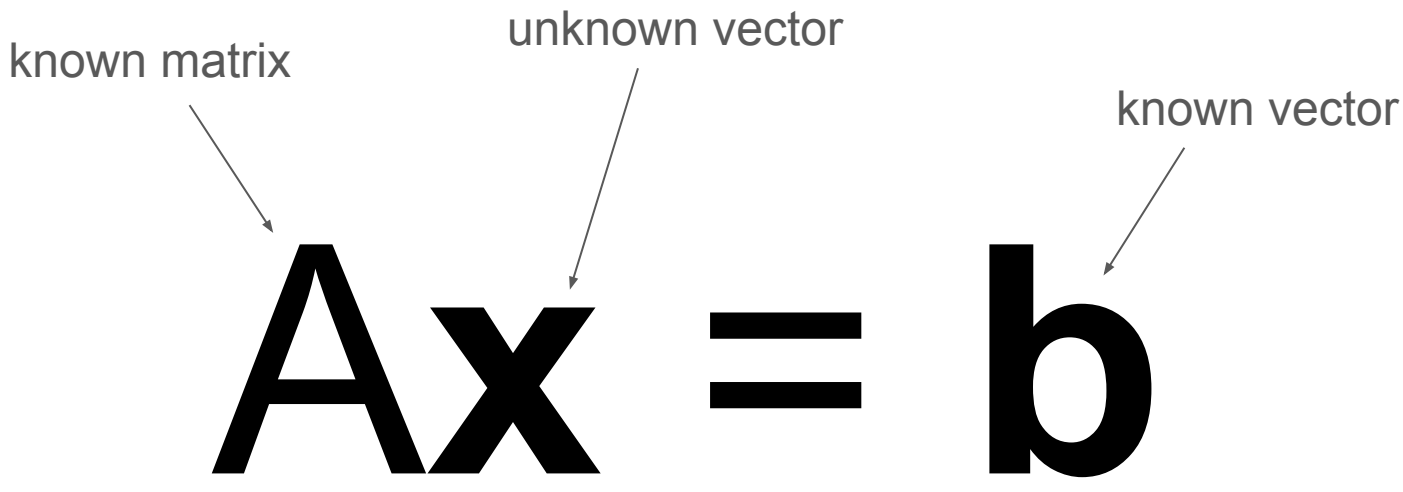
system of linear equations



$$Ax = b$$

system of linear equations

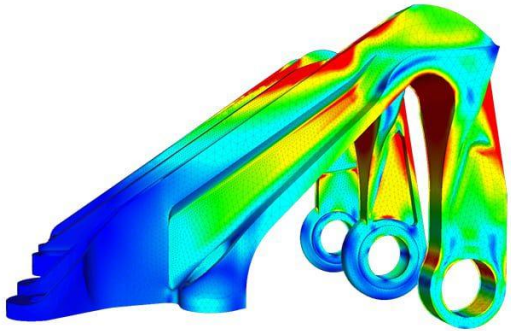
system of linear equations



system of linear equations

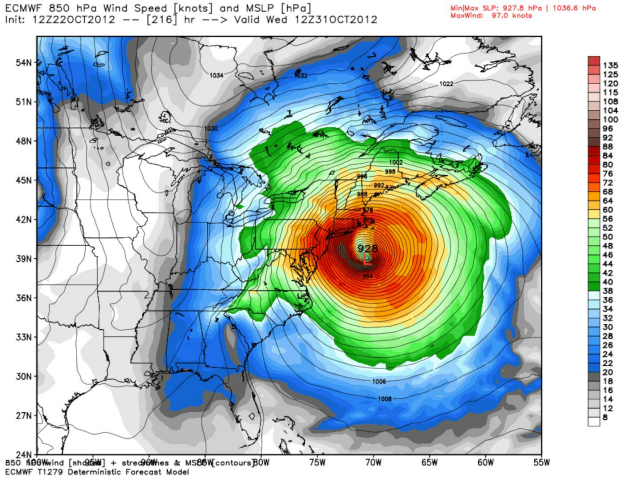
A**x** = **b** is everywhere

A**x** = **b** is everywhere

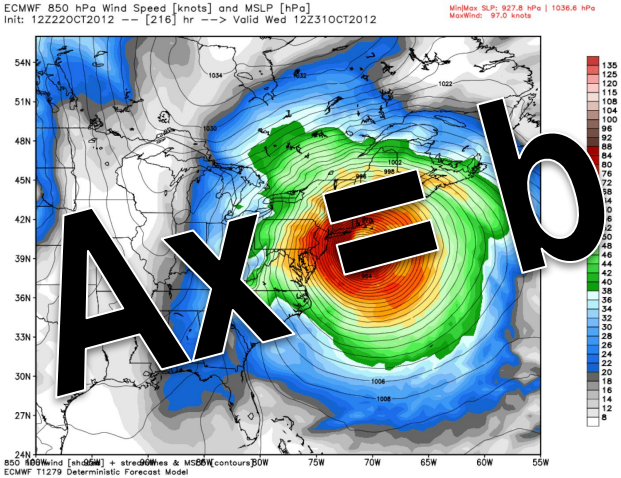


A**x** = **b** is everywhere

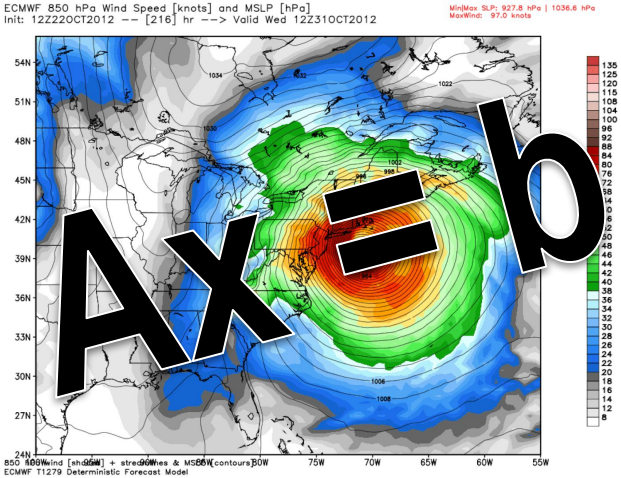
A**x** = **b** is everywhere

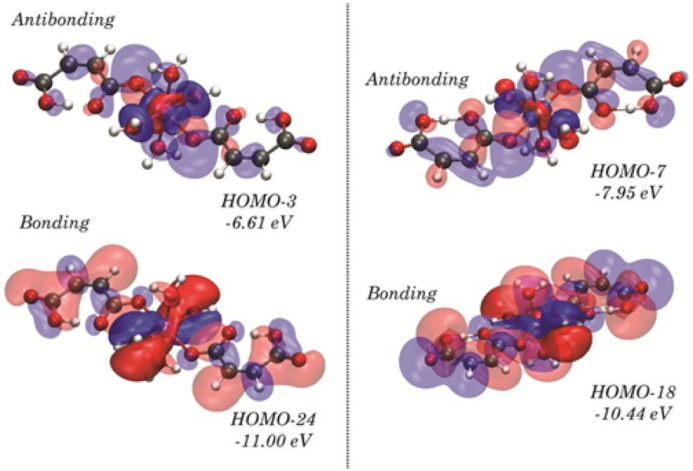


A**x** = **b** is everywhere

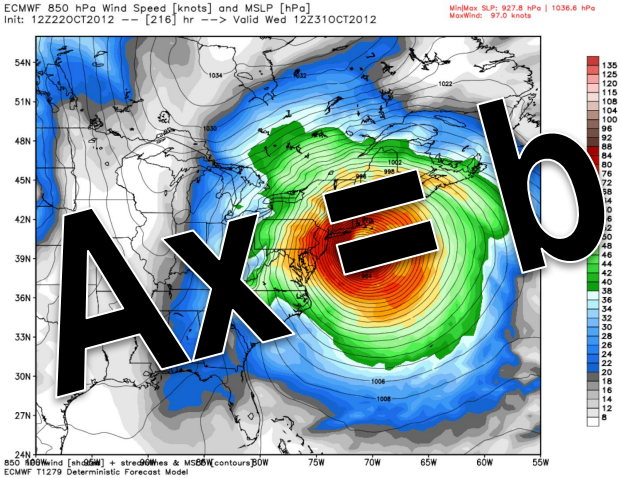


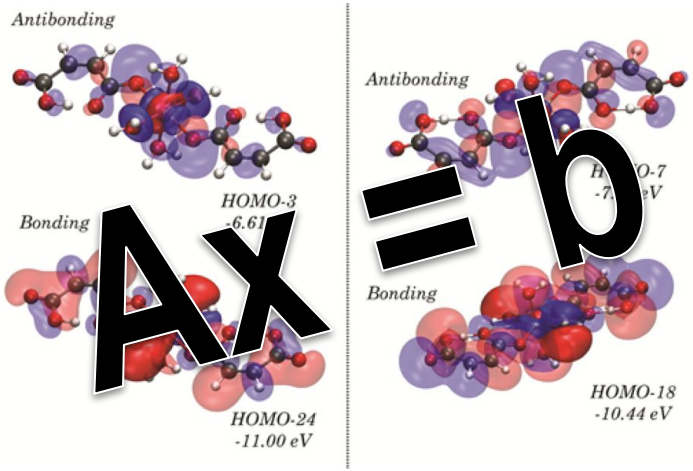
A**x** = **b** is everywhere



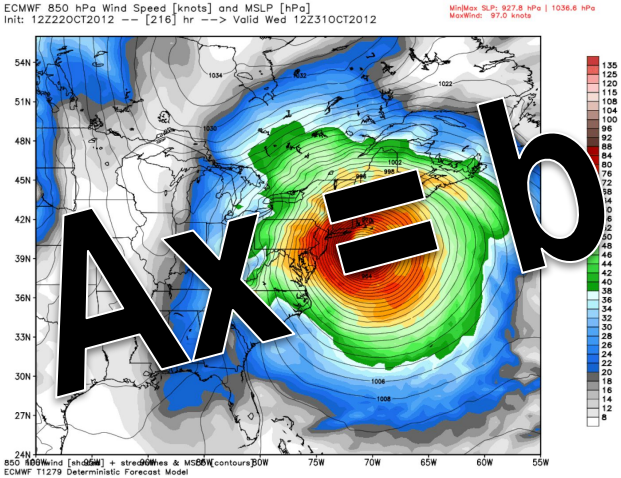


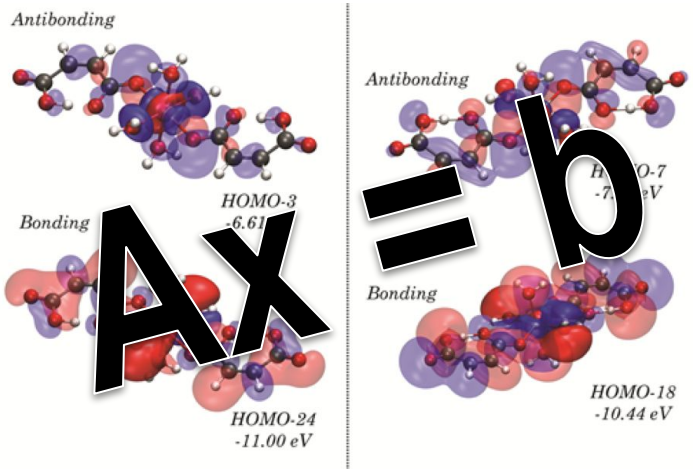
A**x** = **b** is everywhere

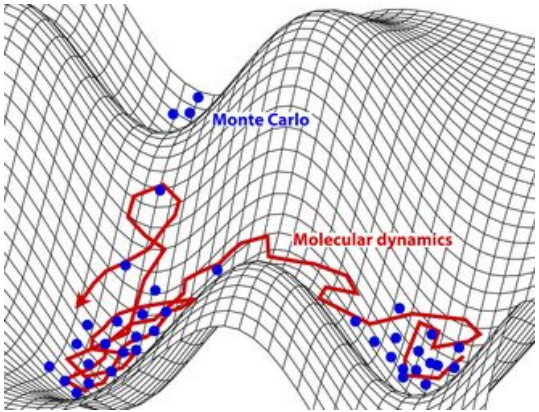




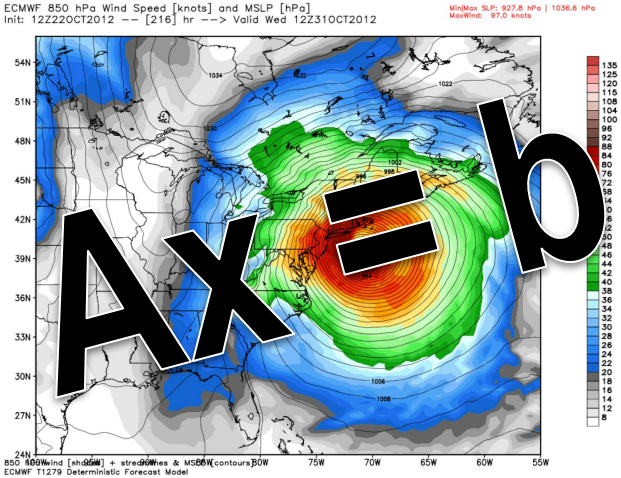
A**x** = **b** is everywhere

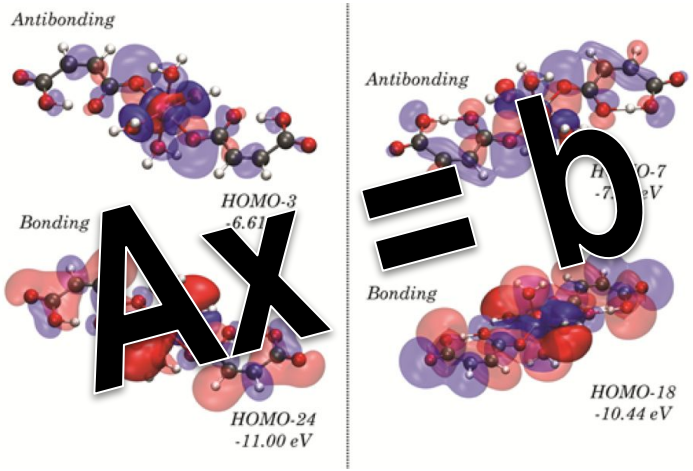


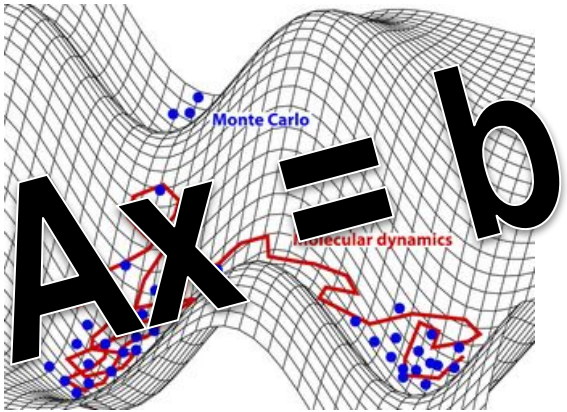




A**x** = **b** is everywhere







A**x** = **b** is a matter of national pride

A**x** = **b** is a matter of national pride

A**x** = **b** is a matter of national pride

TOP 500

A**x** = **b** is a matter of national pride

TOP500

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**El Capitan** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
2	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	**JUPITER Booster** - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	793.40	930.00	13,088
5	**Eagle** - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	

A**x** = **b** has been done before

A**x** = **b** has been done before

A**x** = **b** has been done before

A**x** = **b** has been done before

… but A**x** = **b** can be done **better**

… but A**x** = **b** can be done **better**

- **Parallel composition:**

  What if I want to interleave solving A**x** = **b** with other work?

… but A**x** = **b** can be done **better**

- **Parallel composition:**

  What if I want to interleave solving A**x** = **b** with other work?

- **Format genericity:**

  What if my matrix A is stored in a format you don’t support?

… but A**x** = **b** can be done **better**

- **Parallel composition:**
  What if I want to interleave solving A**x** = **b** with other work?

- **Format genericity:**
  What if my matrix A is stored in a format you don’t support?

- **Partitioning-awareness:**
  What if I want to try out different data partitioning strategies?

… but A**x** = **b** can be done **better**

- **Parallel composition:**
  What if I want to interleave solving A**x** = **b** with other work?

- **Format genericity:**
  What if my matrix A is stored in a format you don't support?

- **Partitioning-awareness:**
  What if I want to try out different data partitioning strategies?

- **Layout flexibility:**
  What if my matrix A lives in more than one place?

… but A**x** = **b** can be done **better**

- **Parallel composition: task parallelism**
  What if I want to interleave solving A**x** = **b** with other work?

- **Format genericity:**
  What if my matrix A is stored in a format you don't support?

- **Partitioning-awareness:**
  What if I want to try out different data partitioning strategies?

- **Layout flexibility:**
  What if my matrix A lives in more than one place?

… but A**x** = **b** can be done **better**

- **Parallel composition: task parallelism (thanks to Legion)**
  What if I want to interleave solving A**x** = **b** with other work?

- **Format genericity:**
  What if my matrix A is stored in a format you don't support?

- **Partitioning-awareness:**
  What if I want to try out different data partitioning strategies?

- **Layout flexibility:**
  What if my matrix A lives in more than one place?

… but A**x** = **b** can be done **better**

- **Parallel composition: task parallelism (thanks to Legion)**
  What if I want to interleave solving A**x** = **b** with other work?

- **Format genericity: *K*, *D*, *R* decomposition of storage formats**
  What if my matrix A is stored in a format you don't support?

- **Partitioning-awareness:**
  What if I want to try out different data partitioning strategies?

- **Layout flexibility:**
  What if my matrix A lives in more than one place?

… but A**x** = **b** can be done **better**

- **Parallel composition: task parallelism (thanks to Legion)**
  What if I want to interleave solving A**x** = **b** with other work?

- **Format genericity: *K*, *D*, *R* decomposition of storage formats**
  What if my matrix A is stored in a format you don't support?

- **Partitioning-awareness: dependent partitioning**
  What if I want to try out different data partitioning strategies?

- **Layout flexibility:**
  What if my matrix A lives in more than one place?

… but A**x** = **b** can be done **better**

- **Parallel composition: task parallelism (thanks to Legion)**
  What if I want to interleave solving A**x** = **b** with other work?

- **Format genericity: *K*, *D*, *R* decomposition of storage formats**
  What if my matrix A is stored in a format you don't support?

- **Partitioning-awareness: dependent partitioning**
  What if I want to try out different data partitioning strategies?

- **Layout flexibility: multi-operator systems**
  What if my matrix A lives in more than one place?

Sparse Matrix Storage Formats

Sparse Matrix Storage Formats

COO $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$

entry	*row*	*col*
3	0	0
2	0	2
4	1	1
5	2	1
1	2	2

Sparse Matrix Storage Formats

COO $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$

entry	*row*	*col*
3	0	0
2	0	2
4	1	1
5	2	1
1	2	2

CSR $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$

entry	*col*
3	0
2	2
4	1
5	1
1	2

rowptr
[0, 1]
[2, 2]
[3, 4]

Sparse Matrix Storage Formats

COO $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$

entry	*row*	*col*
3	0	0
2	0	2
4	1	1
5	2	1
1	2	2

CSR $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$

entry	*col*
3	0
2	2
4	1
5	1
1	2

rowptr
[0, 1]
[2, 2]
[3, 4]

ELL $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$

entry	
3	2
4	0
5	1

col	
1	3
2	0
2	3

Sparse Matrix Storage Formats

COO $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & & \end{bmatrix}$ **CSR** $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \end{bmatrix}$ **ELL** $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ & & 1 \end{bmatrix}$

Key Insight: all sparse matrix storage formats are defined by binary relations between three fundamental index spaces:

entry		
3		
2		
4	1	1
5	2	1
1	2	2

4	1
5	1
1	2

[3, 4]

5	1

	l
	3
	0
2	3

Sparse Matrix Storage Formats

COO $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & & \end{bmatrix}$ **CSR** $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \end{bmatrix}$ **ELL** $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ & & 1 \end{bmatrix}$

Key Insight: all sparse matrix storage formats are defined by binary relations between three fundamental index spaces:

- Kernel index space *K*: nonzero entries of matrix

entry		
3		
2		
4	1	1
5	2	1
1	2	2

4	1
5	1
1	2

[3, 4]

5	1

l	
	3
	0
2	3

Sparse Matrix Storage Formats

COO $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & & \end{bmatrix}$ **CSR** $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \end{bmatrix}$ **ELL** $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ & & 1 \end{bmatrix}$

Key Insight: all sparse matrix storage formats are defined by binary relations between three fundamental index spaces:

- Kernel index space *K*: nonzero entries of matrix
- Domain index space *D*: indices of input vector (columns)

entry		
3		
2		
4	1	1
5	2	1
1	2	2

4	1
5	1
1	2

[3, 4]

	3
	0
5	1
2	3

Sparse Matrix Storage Formats

COO $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & & \end{bmatrix}$ **CSR** $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \end{bmatrix}$ **ELL** $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ & & 1 \end{bmatrix}$

Key Insight: all sparse matrix storage formats are defined by binary relations between three fundamental index spaces:

- Kernel index space *K*: nonzero entries of matrix
- Domain index space *D*: indices of input vector (columns)
- Range index space *R*: indices of output vector (rows)

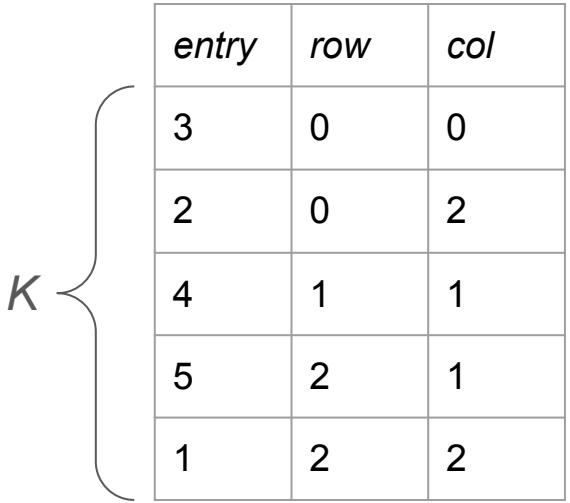
entry		
3		
2		
4	1	1
5	2	1
1	2	2

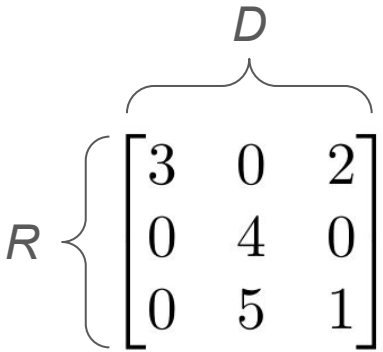
4	1
5	1
1	2

[3, 4]

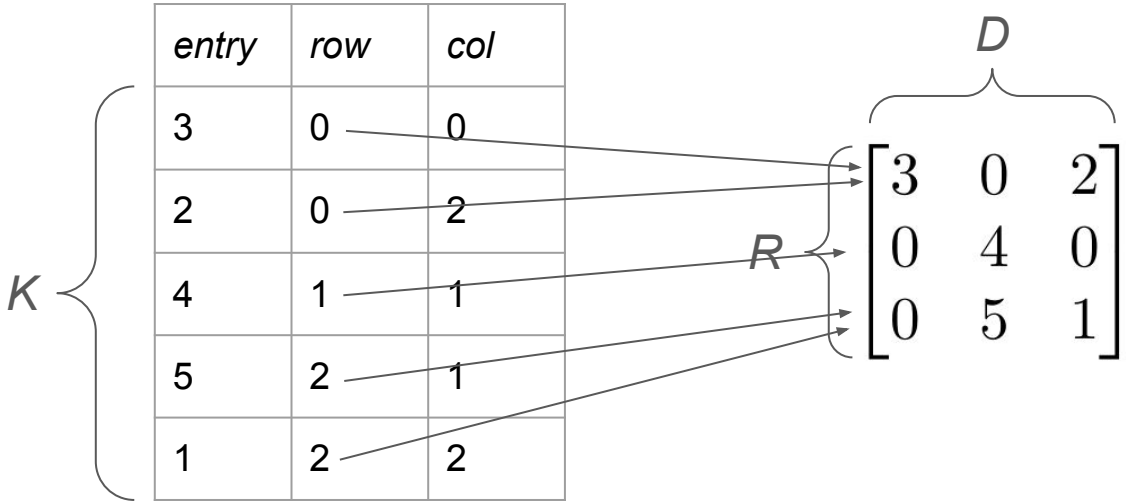
	3
	0
5	1
2	3

COO Sparse Matrix Format

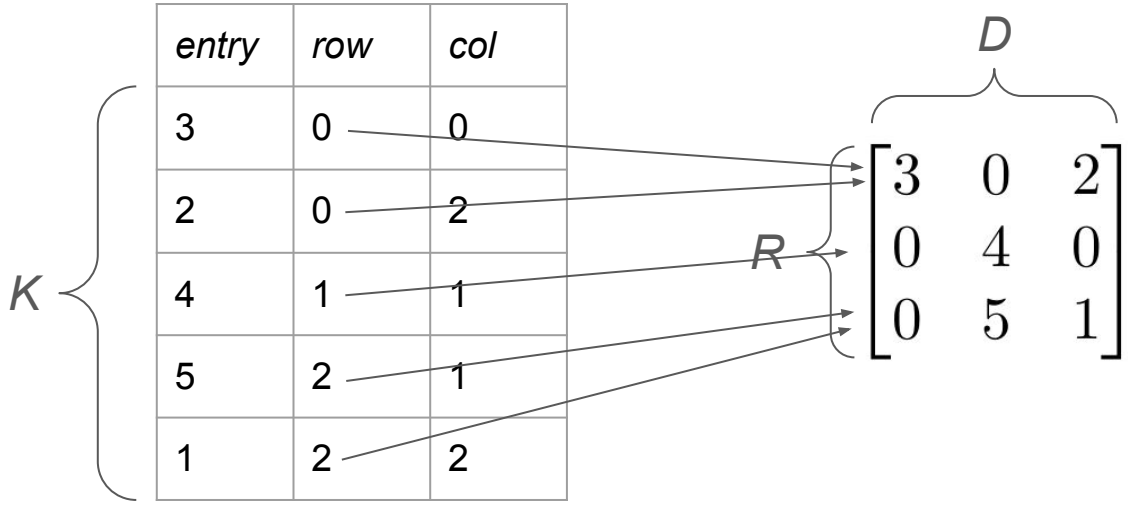




COO Sparse Matrix Format

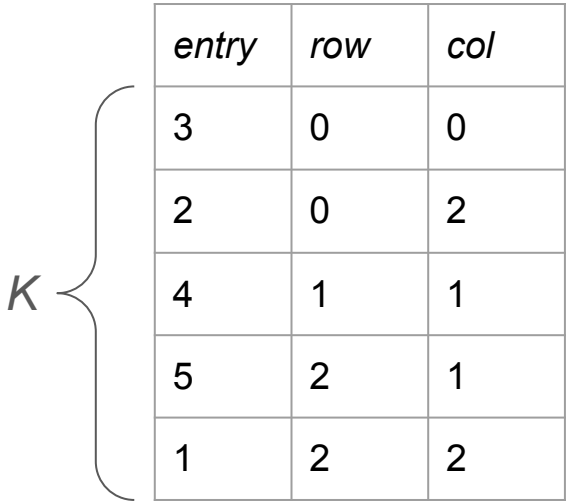


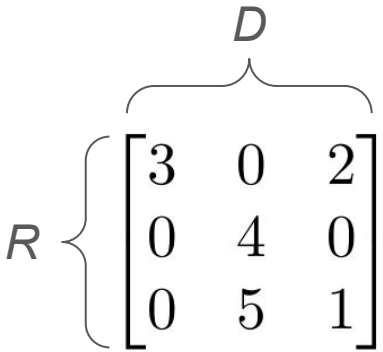
COO Sparse Matrix Format



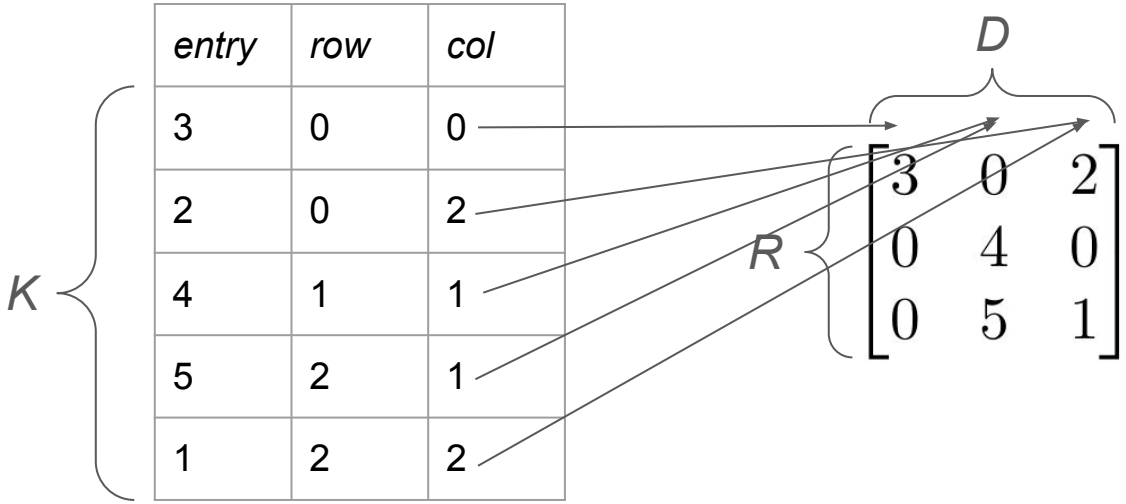
row: K → R

COO Sparse Matrix Format

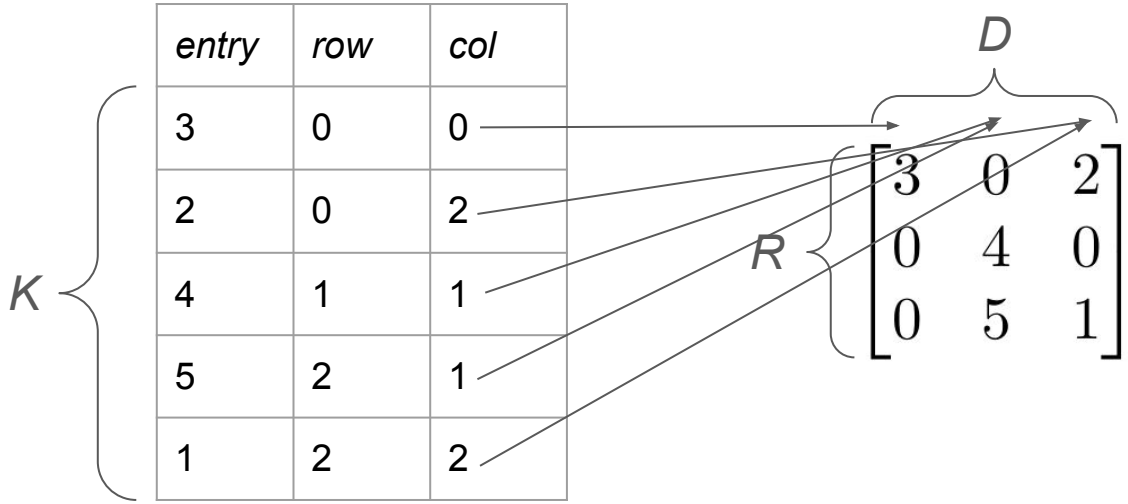




COO Sparse Matrix Format



COO Sparse Matrix Format



COO Sparse Matrix Format

K

entry	*row*	*col*
3	0	0
2	0	2
4	1	1
5	2	1
1	2	2

D

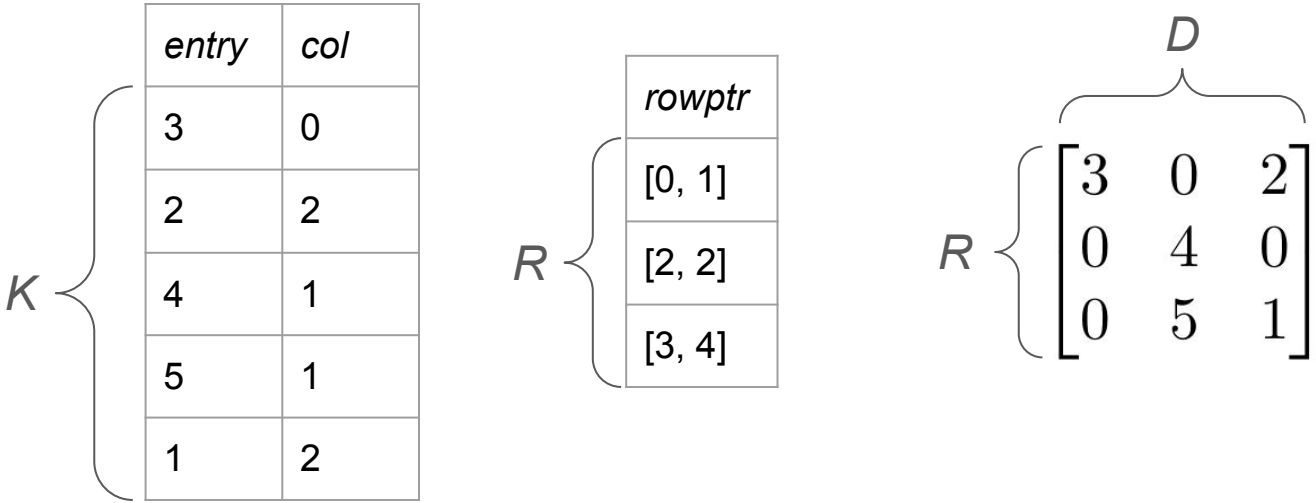
R

$$\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$$

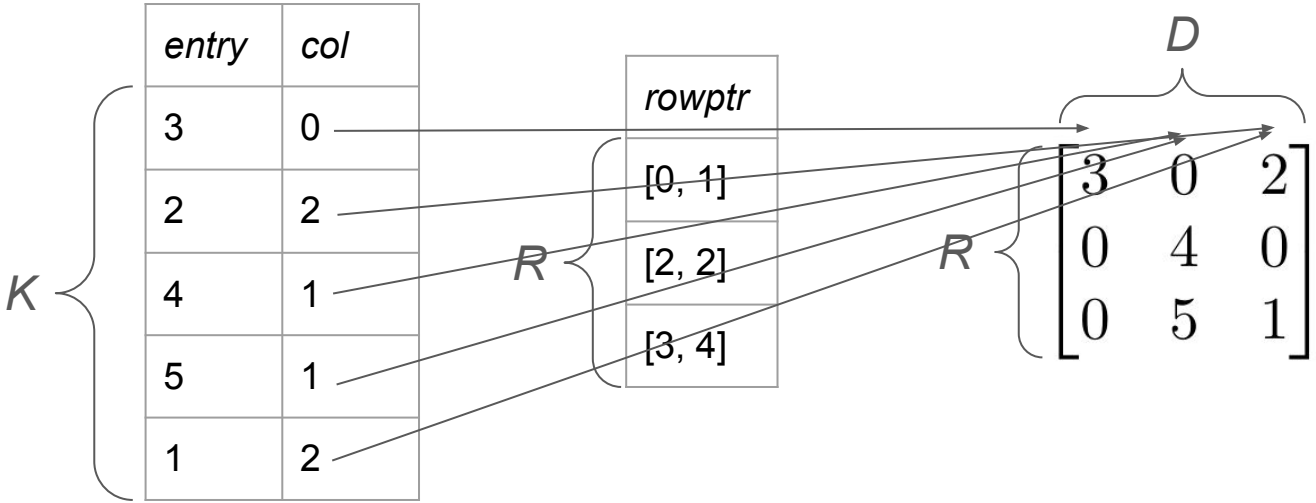
COO format is defined by:

- **column relation** *col*: $K \rightarrow D$
- **row relation** *row*: $K \rightarrow R$

CSR Sparse Matrix Format

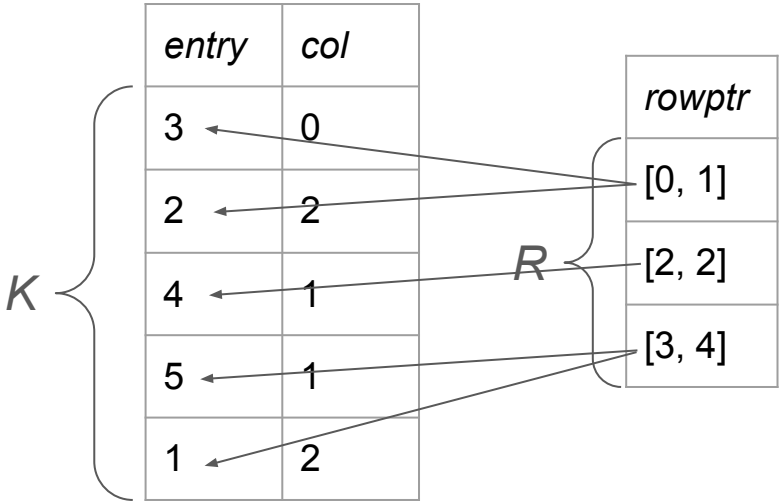


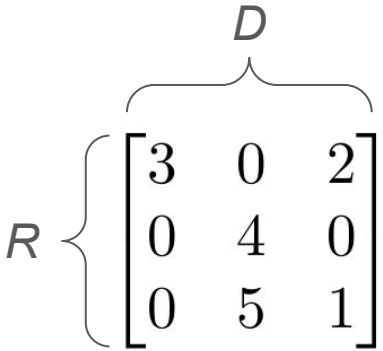
CSR Sparse Matrix Format



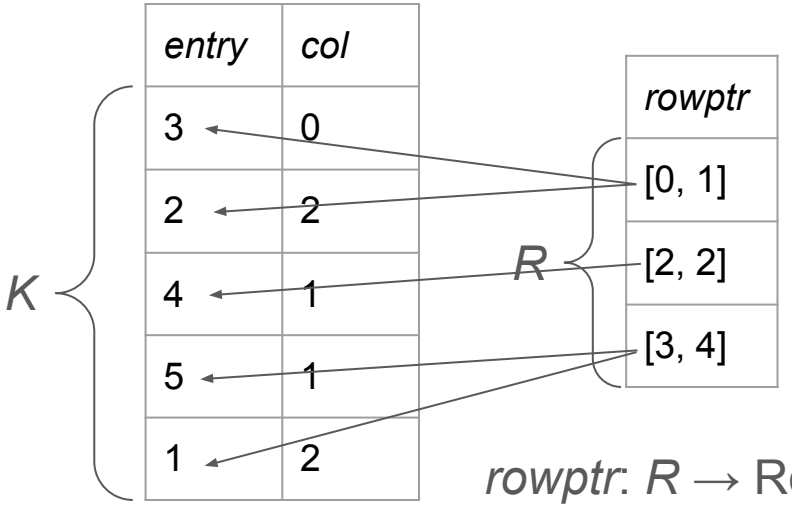
col: K → D

CSR Sparse Matrix Format

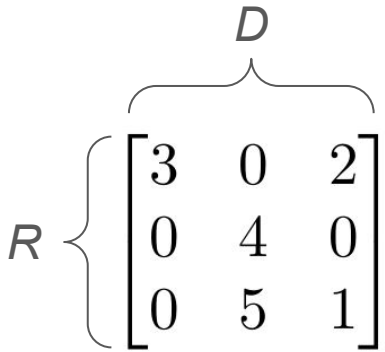




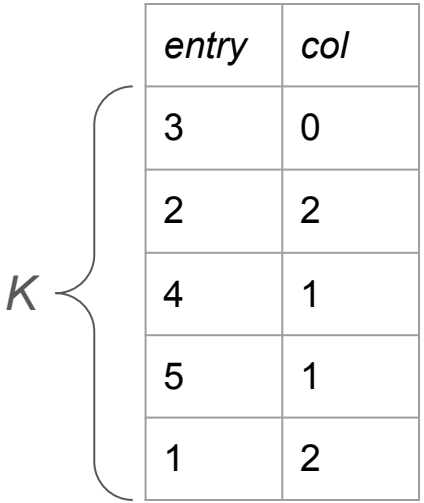
CSR Sparse Matrix Format

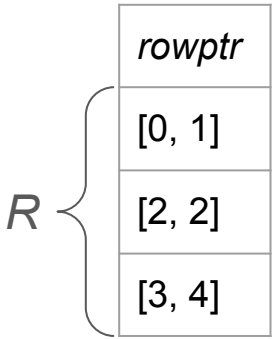


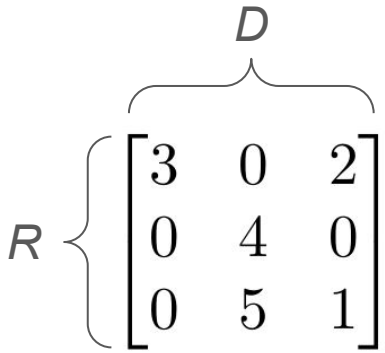
$rowptr: R \rightarrow \text{Rect}[K]$



CSR Sparse Matrix Format



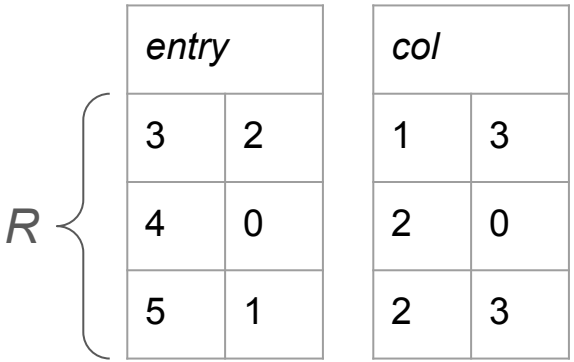


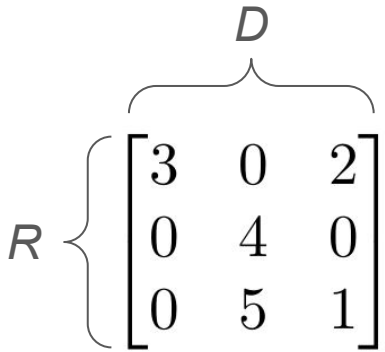


CSR format is defined by:

- **column relation** *col*: $K \rightarrow D$
- **row relation** *rowptr*: $R \rightarrow$ Rect[K]

ELL Sparse Matrix Format

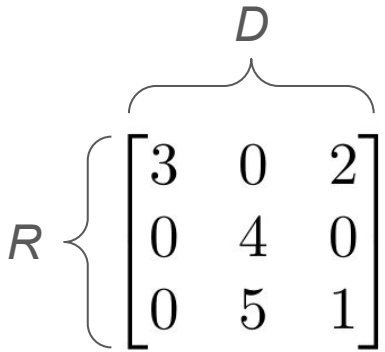




ELL Sparse Matrix Format

$K = R \times N$

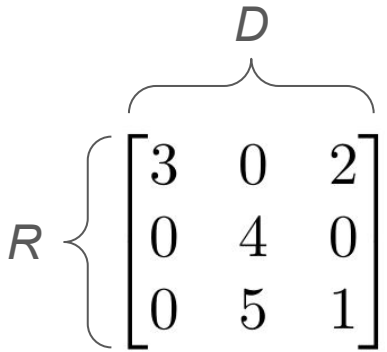
	entry		*col*	
R	3	2	1	3
	4	0	2	0
	5	1	2	3



ELL Sparse Matrix Format

$K = R \times N$

	entry		*col*	
R	3	2	1	3
	4	0	2	0
	5	1	2	3



ELL format is defined by:

- column relation *col*: $K \rightarrow D$
- **implicit** row relation π_1: $R \times N \rightarrow R$

K, *D*, *R* Storage Formats

Format	Structural Assumptions	Column Relation	Row Relation
Dense	$K = R \times D$	$\pi_2 : R \times D \to D$ (implicit)	$\pi_1 : R \times D \to R$ (implicit)
COO	(none)	$\mathsf{col} : K \to D$	$\mathsf{row} : K \to R$
CSR	K is totally ordered	$\mathsf{col} : K \to D$	$\mathsf{rowptr} : R \to [K, K]$
CSC	K is totally ordered	$\mathsf{colptr} : D \to [K, K]$	$\mathsf{row} : K \to R$
ELL	$K = R \times K_0$	$\mathsf{col} : K \to D$	$\pi_1 : R \times K_0 \to R$ (implicit)
ELL′	$K = D \times K_0$	$\pi_1 : D \times K_0 \to D$ (implicit)	$\mathsf{row} : K \to R$
DIA	$D = \{1, \ldots, d\}$ $R = \{1, \ldots, r\}$ $K = K_0 \times \{1, \ldots, d\}$ $\mathsf{offset} : K_0 \to \mathbb{Z}$	$\mathsf{col} : (k_0, i) \mapsto i$ (implicit)	$\mathsf{row} : (k_0, i) \mapsto i - \mathsf{offset}(k_0)$ (implicit)
BCSR	$K = K_0 \times B_R \times B_D$ $D = D_0 \times B_D$ $R = R_0 \times B_R$ K_0 is totally ordered	$\mathsf{col} : K_0 \to D_0$	$\mathsf{rowptr} : R_0 \to [K_0, K_0]$
BCSC	$K = K_0 \times B_R \times B_D$ $D = D_0 \times B_D$ $R = R_0 \times B_R$ K_0 is totally ordered	$\mathsf{colptr} : D_0 \to [K_0, K_0]$	$\mathsf{row} : K_0 \to R_0$

Fig. 3. A wide variety of sparse matrix storage formats are realized in KDRSolvers as particular forms of column and row relations, subject to certain structural assumptions on the index spaces (K, D, R). Some column and row relations, marked as (implicit), can be specified without additional metadata as a consequence of structural assumptions or other auxiliary data. Here, π_i denotes the canonical projection function from a Cartesian product onto its ith coordinate, and whenever an index space K is assumed to be totally ordered, we denote by $[K, K]$ the set of contiguous intervals in K.

K, *D*, *R* Storage Formats

To define a matrix storage format, you give me:

K, *D*, *R* Storage Formats

To define a matrix storage format, you give me:

- **kernel region** containing nonzero entries

K, D, R Storage Formats

To define a matrix storage format, you give me:

- **kernel region** containing nonzero entries
- **binary relations** $K \leftrightarrow D$ and $K \leftrightarrow R$

K, D, R Storage Formats

To define a matrix storage format, you give me:

- **kernel region** containing nonzero entries
- **binary relations** $K \leftrightarrow D$ and $K \leftrightarrow R$
- **task** (function pointer) for matrix-vector multiplication

K, *D*, *R* Storage Formats

To define a matrix storage format, you give me:

- **kernel region** containing nonzero entries
- **binary relations** $K \leftrightarrow D$ and $K \leftrightarrow R$
- **task** (function pointer) for matrix-vector multiplication
  - which can be **automatically derived** from the row and column relations

K, *D*, *R* Storage Formats

To define a matrix storage format, you give me:

- **kernel region** containing nonzero entries
- **binary relations** $K \leftrightarrow D$ and $K \leftrightarrow R$
- **task** (function pointer) for matrix-vector multiplication
  - which can be **automatically derived** from the row and column relations

Then, we can do automatic **dependent partitioning**:

K, *D*, *R* Storage Formats

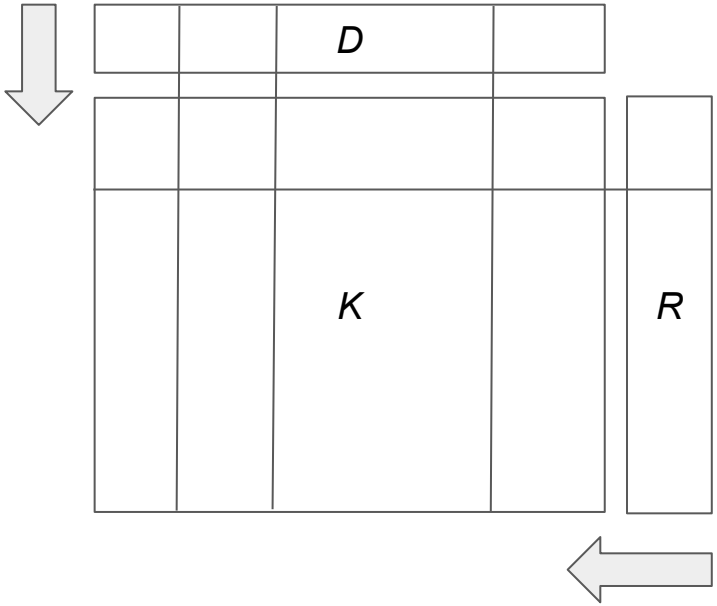
To define a matrix storage format, you give me:

- **kernel region** containing nonzero entries
- **binary relations** $K \leftrightarrow D$ and $K \leftrightarrow R$
- **task** (function pointer) for matrix-vector multiplication
  - which can be **automatically derived** from the row and column relations

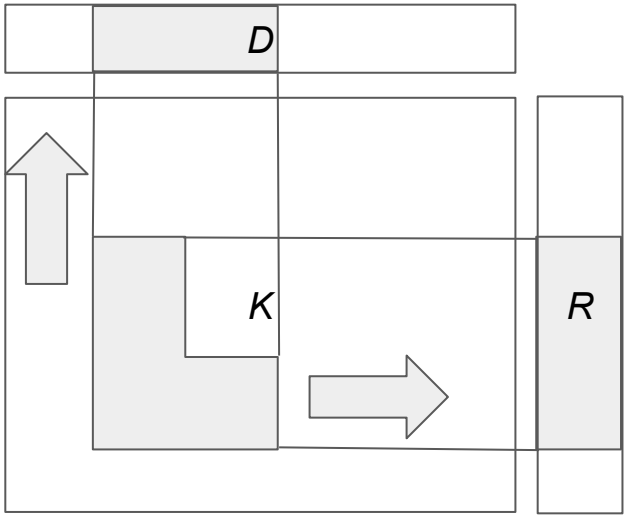
Then, we can do automatic **dependent partitioning**:

transfer a partition of one of (*K*, *D*, *R*) to the others by images/preimages

Matrix Dependent Partitioning Operators



Kernel partition from
domain and range partitions



Domain and range partitions
from kernel partition

Matrix Dependent Partitioning Operators

```
virtual Legion::IndexPartition
create_kernel_partition_from_domain_partition(
    Legion::IndexPartition domain_partition
) const = 0;

virtual Legion::IndexPartition create_kernel_partition_from_range_partition(
    Legion::IndexPartition range_partition
) const = 0;

virtual Legion::IndexPartition
create_domain_partition_from_kernel_partition(
    Legion::IndexSpace domain_space, Legion::IndexPartition kernel_partition
) const = 0;

virtual Legion::IndexPartition create_range_partition_from_kernel_partition(
    Legion::IndexSpace range_space, Legion::IndexPartition kernel_partition
) const = 0;
```

Multi-Operator Systems

Who says A**x** = **b** only involves a single matrix A and vector **b**?

Multi-Operator Systems

Who says A**x** = **b** only involves a single matrix A and vector **b**?

$$A_{11}\,\mathbf{x}_1 + A_{12}\,\mathbf{x}_2 + A_{13}\,\mathbf{x}_3 = \mathbf{b}_1$$
$$A_{21}\,\mathbf{x}_1 + A_{22}\,\mathbf{x}_2 + A_{23}\,\mathbf{x}_3 = \mathbf{b}_2$$

Multi-Operator Systems

Who says A**x** = **b** only involves a single matrix A and vector **b**?

$$A_{11}\,\mathbf{x}_1 + A_{12}\,\mathbf{x}_2 + A_{13}\,\mathbf{x}_3 = \mathbf{b}_1$$
$$A_{21}\,\mathbf{x}_1 + A_{22}\,\mathbf{x}_2 + A_{23}\,\mathbf{x}_3 = \mathbf{b}_2$$

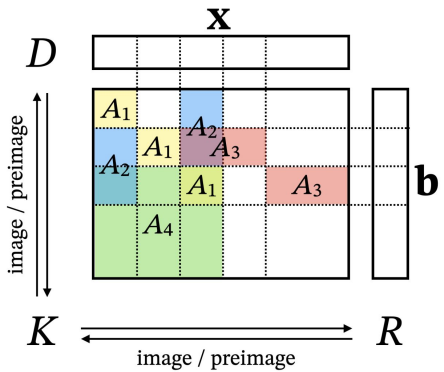
We allow a *logical* matrix or vector
to consist of multiple *physical* pieces

Multi-Operator Systems

Who says A**x** = **b** only involves a single matrix A and vector **b**?

$$A_{11}\,\mathbf{x}_1 + A_{12}\,\mathbf{x}_2 + A_{13}\,\mathbf{x}_3 = \mathbf{b}_1$$
$$A_{21}\,\mathbf{x}_1 + A_{22}\,\mathbf{x}_2 + A_{23}\,\mathbf{x}_3 = \mathbf{b}_2$$

We allow a *logical* matrix or vector
to consist of multiple *physical* pieces



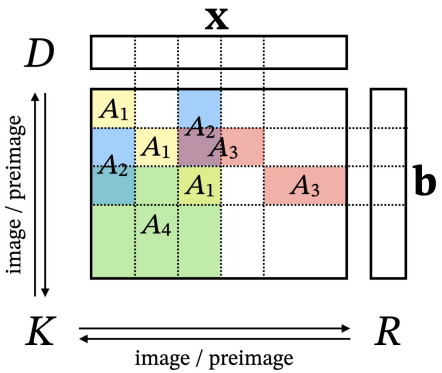
Multi-Operator Systems

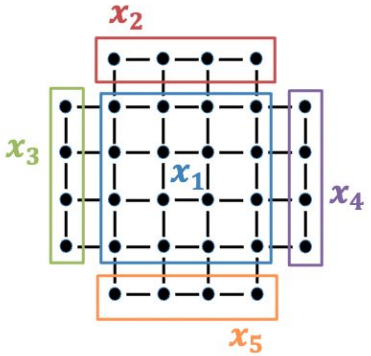
Who says A**x** = **b** only involves a single matrix A and vector **b**?

$$A_{11}\,\mathbf{x}_1 + A_{12}\,\mathbf{x}_2 + A_{13}\,\mathbf{x}_3 = \mathbf{b}_1$$
$$A_{21}\,\mathbf{x}_1 + A_{22}\,\mathbf{x}_2 + A_{23}\,\mathbf{x}_3 = \mathbf{b}_2$$

We allow a *logical* matrix or vector to consist of multiple *physical* pieces

Use data wherever it naturally exists in the context of your application





An example of ***heterogeneous dimension***: a 2D grid with four 1D boundary pieces. Note that we can easily enforce periodic boundary conditions by identifying $x_2 = x_5$ and $x_3 = x_4$.

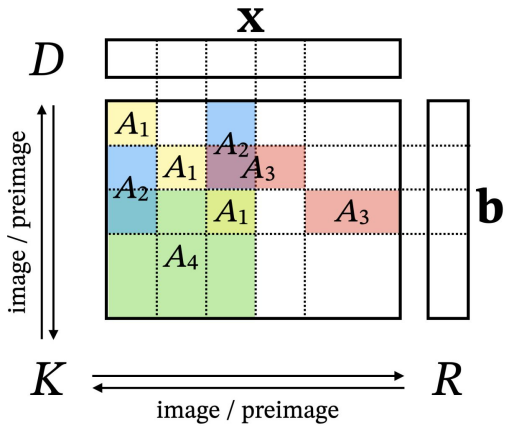
Operations on Multi-Operator Systems

Vector operations (SCAL, AXPY, DOT) are done component-by-component

Operations on Multi-Operator Systems

Vector operations (SCAL, AXPY, DOT) are done component-by-component

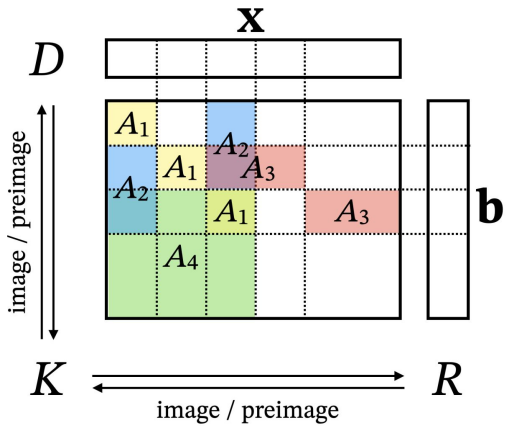
For matrix-vector products, we launch a separate task for each matrix piece and reduce into a single vector



Operations on Multi-Operator Systems

Vector operations (SCAL, AXPY, DOT) are done component-by-component

For matrix-vector products, we launch a separate task for each matrix piece and reduce into a single vector



These are the only two operations needed
for **Krylov subspace methods**: CG, BiCG, MINRES, GMRES, …

Multiple Right-Hand Sides

$A\mathbf{x}_1 = \mathbf{b}_1$
$A\mathbf{x}_2 = \mathbf{b}_2$
$A\mathbf{x}_3 = \mathbf{b}_3$

$$\begin{bmatrix} A & & \\ & A & \\ & & A \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \mathbf{b}_3 \end{bmatrix}$$

Related Systems

$A\mathbf{x}_1 = \mathbf{b}_1$
$A\mathbf{x}_2 = \mathbf{b}_2$
$A\mathbf{x}_3 = \mathbf{b}_3$

$$\begin{bmatrix} A & & \\ & A & \\ & & A \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \mathbf{b}_3 \end{bmatrix}$$

$(A + \delta A_1)\,\mathbf{x}_1 = \mathbf{b}_1$
$(A + \delta A_2)\,\mathbf{x}_2 = \mathbf{b}_2$
$(A + \delta A_3)\,\mathbf{x}_3 = \mathbf{b}_3$

$$\begin{bmatrix} A + \delta A_1 & & \\ & A + \delta A_2 & \\ & & A + \delta A_3 \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \mathbf{b}_3 \end{bmatrix}$$

Implementing Krylov Algorithms

```
template <typename ENTRY_T>
class CGSolver {

    \impname{}::Planner<ENTRY_T> &planner;
    static constexpr vec_id SOL = 0;
    static constexpr vec_id RHS = 1;
    vec_id P;
    vec_id Q;
    vec_id R;
    Scalar<ENTRY_T> res; // squared residual

public:

    explicit CGSolver(Planner<ENTRY_T> &planner)
        : planner(planner) {
        assert(planner.is_square());
        assert(!planner.has_preconditioner());
        P = planner.allocate_workspace_vector();
        Q = planner.allocate_workspace_vector();
        R = planner.allocate_workspace_vector();
        planner.copy(P, RHS);
        planner.copy(R, RHS);
        res = planner.dot(R, R);
    }

    void step() {
        planner.matmul(Q, P);
        Scalar<ENTRY_T> p_norm = planner.dot(P, Q);
        planner.axpy(SOL, res / p_norm, P);
        planner.axpy(R, -res / p_norm, Q);
        Scalar<ENTRY_T> new_res = planner.dot(R, R);
        planner.xpay(P, new_res, res, R);
        res = new_res;
    }

    Scalar<ENTRY_T> get_convergence_measure() const
    {
        return res;
    }

}; // class CGSolver<ENTRY_T>
```

Implementing Krylov Algorithms

```
template <typename ENTRY_T>
class CGSolver {

    \impname{}::Planner<ENTRY_T> &planner;
    static constexpr vec_id SOL = 0;
    static constexpr vec_id RHS = 1;
    vec_id P;
    vec_id Q;
    vec_id R;
    Scalar<ENTRY_T> res; // squared residual

public:

    explicit CGSolver(Planner<ENTRY_T> &planner)
        : planner(planner) {
        assert(planner.is_square());
        assert(!planner.has_preconditioner());
        P = planner.allocate_workspace_vector();
        Q = planner.allocate_workspace_vector();
        R = planner.allocate_workspace_vector();
        planner.copy(P, RHS);
        planner.copy(R, RHS);
        res = planner.dot(R, R);
    }

    void step() {
        planner.matmul(Q, P);
        Scalar<ENTRY_T> p_norm = planner.dot(P, Q);
        planner.axpy(SOL, res / p_norm, P);
        planner.axpy(R, -res / p_norm, Q);
        Scalar<ENTRY_T> new_res = planner.dot(R, R);
        planner.xpay(P, new_res, res, R);
        res = new_res;
    }

    Scalar<ENTRY_T> get_convergence_measure() const
    {
        return res;
    }

}; // class CGSolver<ENTRY_T>
```

```
void step() {
    planner.matmul(Q, P);
    Scalar<ENTRY_T> p_norm = planner.dot(P, Q);
    planner.axpy(SOL, res / p_norm, P);
    planner.axpy(R, -res / p_norm, Q);
    Scalar<ENTRY_T> new_res = planner.dot(R, R);
    planner.xpay(P, new_res, res, R);
    res = new_res;
}
```

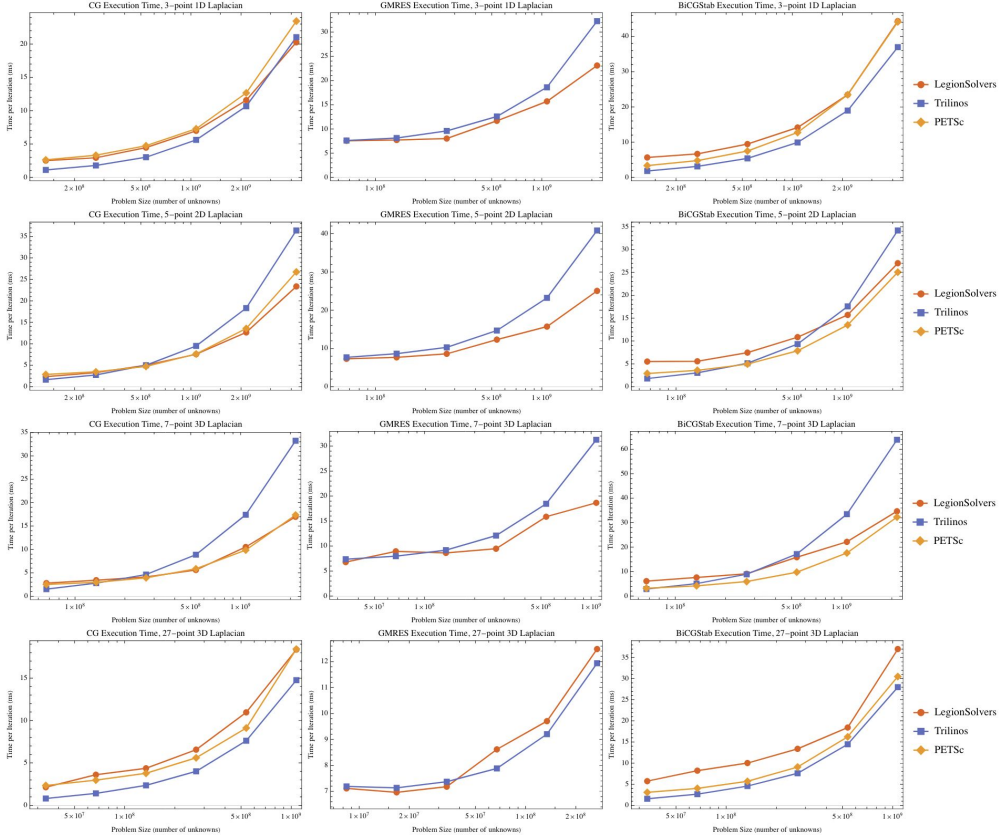
Implementing Krylov Algorithms

```
template <typename ENTRY_T>
class CGSolver {

    \impname{}::Planner<ENTRY_T> &planner;
    static constexpr vec_id SOL = 0;
    static constexpr vec_id RHS = 1;
    vec_id P;
    vec_id Q;
    vec_id R;
    Scalar<ENTRY_T> res; // squared residual

public:

    explicit CGSolver(Planner<ENTRY_T> &planner)
        : planner(planner) {
        assert(planner.is_square());
        assert(!planner.has_preconditioner());
        P = planner.allocate_workspace_vector();
        Q = planner.allocate_workspace_vector();
        R = planner.allocate_workspace_vector();
        planner.copy(P, RHS);
        planner.copy(R, RHS);
        res = planner.dot(R, R);
    }

    void step() {
        planner.matmul(Q, P);
        Scalar<ENTRY_T> p_norm = planner.dot(P, Q);
        planner.axpy(SOL, res / p_norm, P);
        planner.axpy(R, -res / p_norm, Q);
        Scalar<ENTRY_T> new_res = planner.dot(R, R);
        planner.xpay(P, new_res, res, R);
        res = new_res;
    }

    Scalar<ENTRY_T> get_convergence_measure() const
    {
        return res;
    }

}; // class CGSolver<ENTRY_T>
```

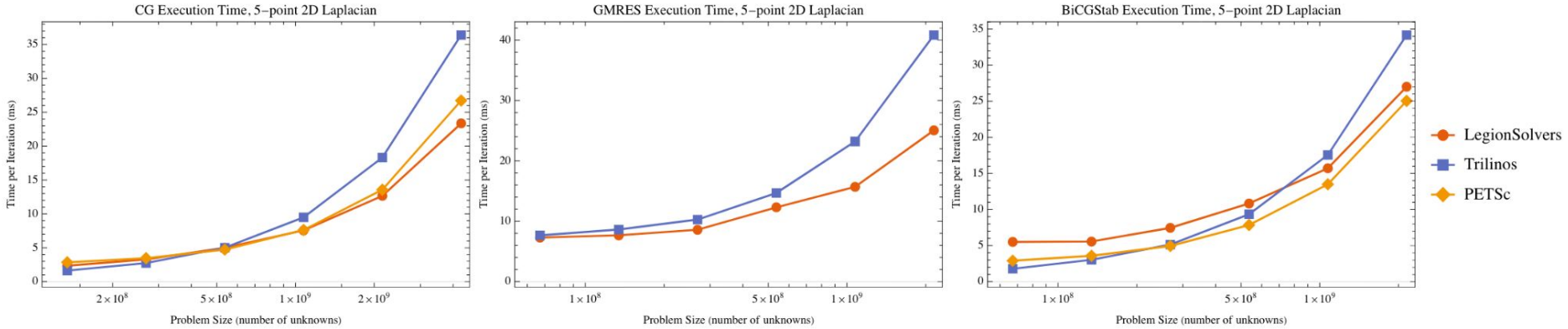
note: solver algorithms do not need to be aware of our new abstractions

```
void step() {
    planner.matmul(Q, P);
    Scalar<ENTRY_T> p_norm = planner.dot(P, Q);
    planner.axpy(SOL, res / p_norm, P);
    planner.axpy(R, -res / p_norm, Q);
    Scalar<ENTRY_T> new_res = planner.dot(R, R);
    planner.xpay(P, new_res, res, R);
    res = new_res;
}
```

Benchmarks: Performance Comparison



Benchmarks: Performance Comparison



Benchmarks: Multi-Operator Systems

$$\begin{bmatrix} \\ A \\ \\ \end{bmatrix} \begin{bmatrix} \\ \mathbf{x} \\ \\ \end{bmatrix} = \begin{bmatrix} \\ \mathbf{b} \\ \\ \end{bmatrix}$$

Benchmarks: Multi-Operator Systems

$$\begin{bmatrix} \quad A \quad \end{bmatrix} \begin{bmatrix} \mathbf{x} \end{bmatrix} = \begin{bmatrix} \mathbf{b} \end{bmatrix}$$

$$\left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array}\right] \left[\begin{array}{c} \mathbf{x}_1 \\ \hline \mathbf{x}_2 \end{array}\right] = \left[\begin{array}{c} \mathbf{b}_1 \\ \hline \mathbf{b}_2 \end{array}\right]$$

Benchmarks: Multi-Operator Systems

$$\begin{bmatrix} & A & \end{bmatrix} \begin{bmatrix} \mathbf{x} \end{bmatrix} = \begin{bmatrix} \mathbf{b} \end{bmatrix}$$

$$\left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array}\right] \left[\begin{array}{c} \mathbf{x}_1 \\ \hline \mathbf{x}_2 \end{array}\right] = \left[\begin{array}{c} \mathbf{b}_1 \\ \hline \mathbf{b}_2 \end{array}\right]$$

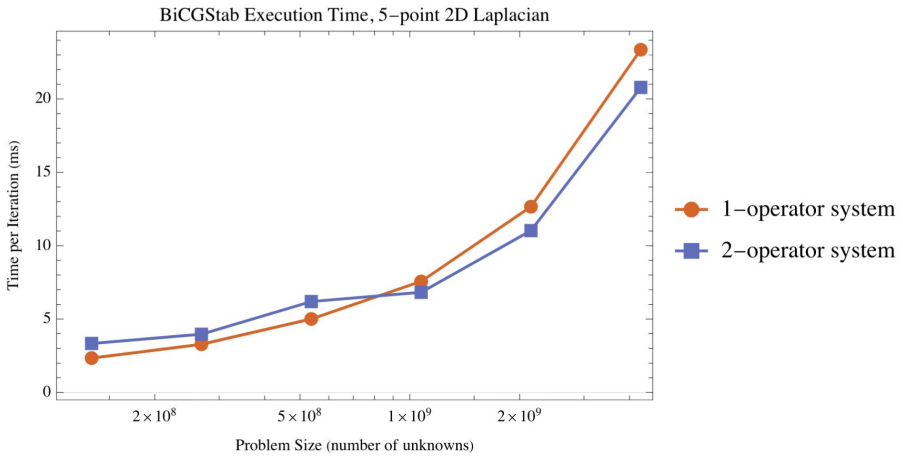


Figure 9: Execution time per iteration of BiCGStab using a 5-point Laplacian stencil on a $2^n \times 2^n$ grid, measured as a function of n, formulated as either a single-operator system (red) or a multi-operator system (blue).

Benchmarks: Dynamic Load-Balancing

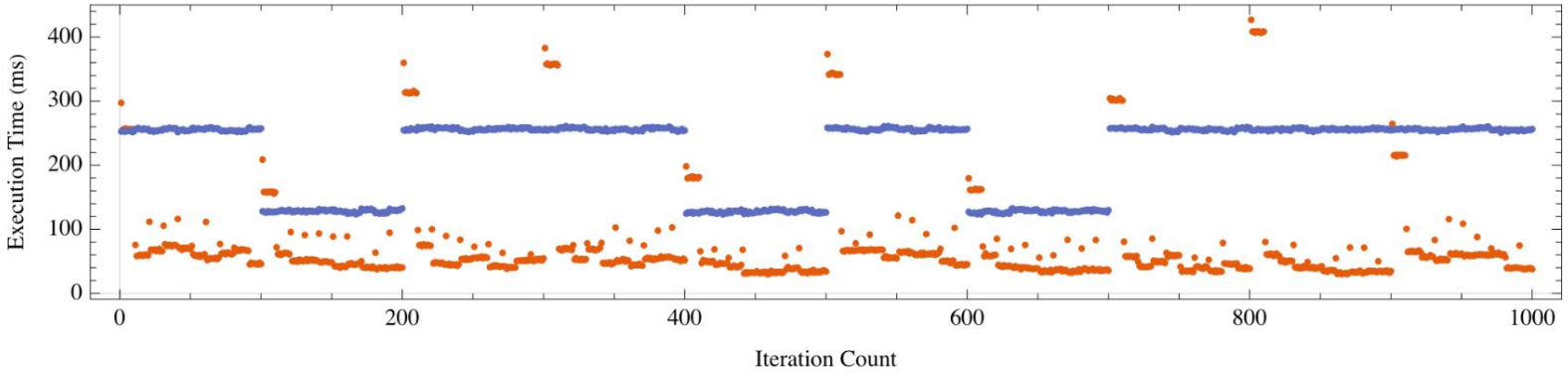


Figure 10: Execution time per iteration of BiCGStab using a 5-point Laplacian stencil on a $2^{16} \times 2^{16}$ grid with (red) and without (blue) dynamic load-balancing.

Benchmarks: Dynamic Load-Balancing

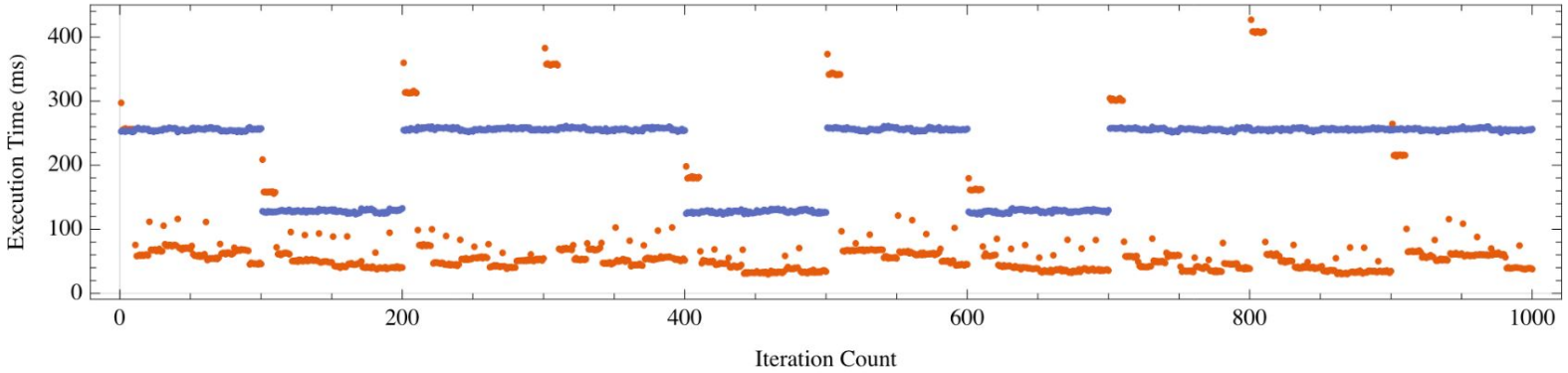


Figure 10: Execution time per iteration of BiCGStab using a 5-point Laplacian stencil on a $2^{16} \times 2^{16}$ grid with (red) and without (blue) dynamic load-balancing.

In addition to a BiCGStab solve, each CPU also runs a dummy task that occupies a uniformly random number of its cores

Benchmarks: Dynamic Load-Balancing

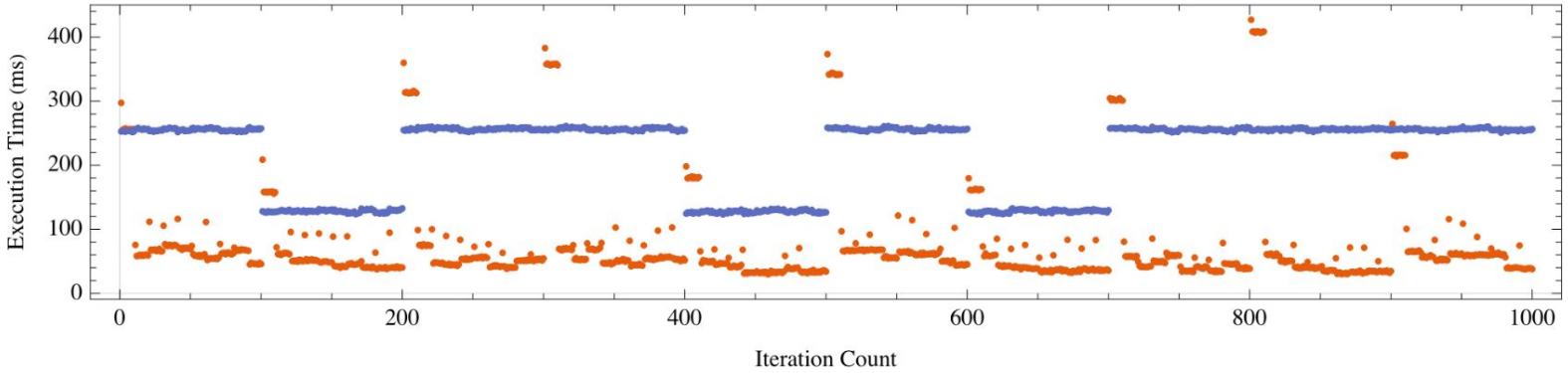


Figure 10: Execution time per iteration of BiCGStab using a 5-point Laplacian stencil on a $2^{16} \times 2^{16}$ grid with (red) and without (blue) dynamic load-balancing.

In addition to a BiCGStab solve, each CPU also runs a dummy task that occupies a uniformly random number of its cores

Number of occupied cores is randomized every 100th CG iteration

Benchmarks: Dynamic Load-Balancing

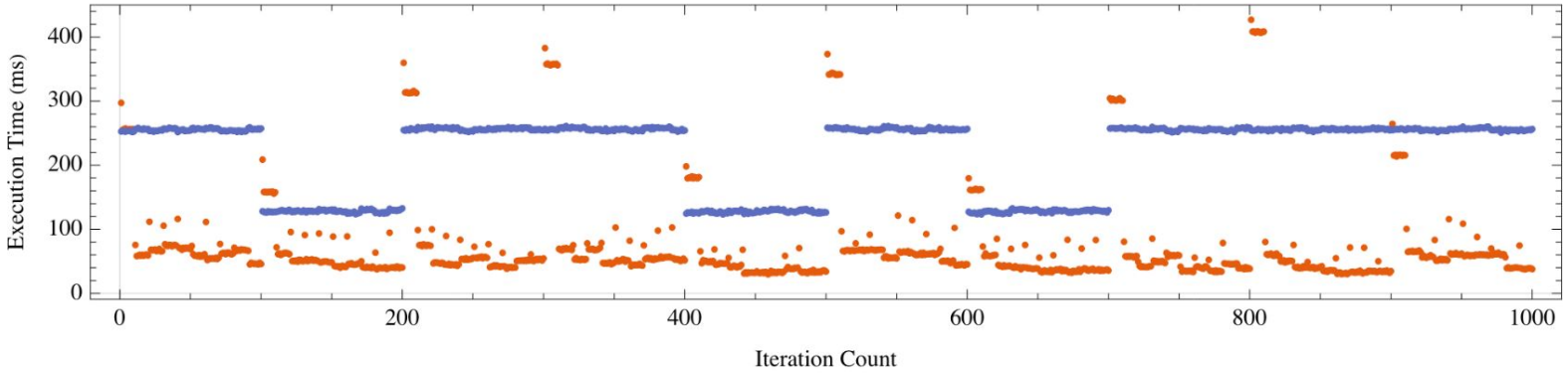


Figure 10: Execution time per iteration of BiCGStab using a 5-point Laplacian stencil on a $2^{16} \times 2^{16}$ grid with (red) and without (blue) dynamic load-balancing.

In addition to a BiCGStab solve, each CPU also runs a dummy task that occupies a uniformly random number of its cores

Number of occupied cores is randomized every 100th CG iteration

We run a dynamic load-balancing strategy every 10th CG iteration

Conclusion

Conclusion

- We have presented a novel approach to designing iterative solver libraries with more **flexibility** and **extensibility** without sacrificing **performance**.

Conclusion

- We have presented a novel approach to designing iterative solver libraries with more **flexibility** and **extensibility** without sacrificing **performance**.

- The algebra of row and column relations allows us to support:
  - *user-defined* matrix storage formats
  - *user-defined* data partitioning strategies

  with **no library modification** and minimal application code changes.

Conclusion

- We have presented a novel approach to designing iterative solver libraries with more **flexibility** and **extensibility** without sacrificing **performance**.

- The algebra of row and column relations allows us to support:
  - *user-defined* matrix storage formats
  - *user-defined* data partitioning strategies

  with **no library modification** and minimal application code changes.

- Multi-operator systems **minimize communication** by processing data *in situ* without explicit copying or matrix/vector assembly steps.